Can we improve this project?

Discussion summary prepared 14 September 2026. Includes the earlier code, memory and novelty reviews so the whole team has one consistent explanation.

Yes. Build on the existing code.

The useful foundations already exist. First make the grid smaller, measure the complete pipeline, and fix how terrain and obstacles are interpreted. A rewrite or a new neural network is not the first step.

What the system is trying to do

LiDAR produces a cloud of 3D points. A segmentation model gives points labels. The grid engine groups points into squares on the ground and stores height and class information. Nearby squares are small; distant squares are larger. A viewer displays the result.

Distance from sensor	Current cell width	Meaning
2.5-10 m	5 cm	Fine detail close to the vehicle
10-25 m	10 cm	Moderate detail
25-50 m	20 cm	Larger regions grouped together
50-100 m	40 cm	Coarsest current representation

The code uses sparse arrays on an aligned, power-of-two lattice. It does not currently implement a scene-adaptive quadtree. Points outside the configured range are excluded; the area inside 2.5 m is not covered by this grid.

What is an elevation map?

Think of a chessboard placed over the ground. A normal occupancy map says whether each square is blocked. An elevation map also says how high the surface is. This is called **2.5D**: a 2D layout carrying height information, rather than a full volume of 3D cubes.

Example: a road cell could have ground height 0.00 m, while a curb cell has ground height 0.15 m. Your code stores height minimum, maximum, mean and variance. However, averaging road and car points together does not give the true ground height. Ground and obstacle evidence need separate treatment.

What is running today?

Sequences 09 and 10 are stored locally: 1,591 and 1,201 scans. The local VNC viewer uses dataset labels. That demonstrates mapping and visualization; it does not demonstrate neural-network inference. Local model weights were not present at this review. No servers, downloads or optimization jobs were started for this report.

Memory: evidence and opportunity

The main question is not how many files we own. It is how much information we keep in memory while processing each scan.

Same sample; map arrays only. Third bar is a projection, not a measured implementation.

Measured sample: 10 evenly spread frames across sequences 09 and 10, using ground-truth labels and the same points within a planar radius of 2.5-100 m. Both elevation grids store the same statistics. Average memory was **5.03 MB uniform versus 3.86 MB foveated: 23.15% lower**. This was an exploratory sample, not a full-sequence or live-inference benchmark. [L1]

The existing minimal sparse 3D voxel baseline used about **1.31 MB** on that sample. It stores much less information per element. Keep it in the comparison, but do not claim the current elevation grid is smaller than it.

Measure	Plain-language meaning	What to report
Stored map memory	The information retained in the finished grid.	Array bytes after all required layers exist.
Peak RAM / GPU memory	The largest working footprint, including model, input, temporary arrays and queues.	Measure the process and GPU separately.
Dataset disk space	Saved scans, labels and other files.	Report separately; do not call it runtime memory.

The first practical reduction: smaller cell records

A current finished cell stores **84 bytes**. A proposed layout uses 32-bit coordinates and counts, four 32-bit height statistics, four 32-bit semantic counts, and three 32-bit derived values: **56 bytes per cell**. That is one-third less cell storage at the same cell count.

Example: applying 56/84 to the measured 3.86 MB gives approximately **2.58 MB**. This is an estimate, not an achieved result. Keep precise working accumulators where necessary, check integer bounds, and measure height/variance error after packing. Current bit-for-bit numerical guarantees would need revision.

Why the old 3.30 MB headline needs context

The benchmark measures the grid before adding its three derived arrays: **72 bytes per cell**, rather than the finished 84. The historical 3.30 MB result therefore must not be described as the full analytics payload. Array accounting also excludes construction peaks, model memory and viewer overhead. [L2]

Fix the bottlenecks in order

These are extensions to the current project. Start with changes that can be measured on the two downloaded sequences.

Priority / change	Why it helps	What must still hold
1. Correct measurements	Compare equivalent uniform and foveated grids after all layers are added. Measure peak RAM separately.	Same points, labels, range, hardware and required outputs.
2. Compact cell records	Reduce retained bytes per cell; compute duplicate derived values only when needed.	Bound numerical error and integer overflow; keep obstacle evidence.
3. Build final cells directly	A point at 70 m should enter a 40 cm cell directly, without 5/10/20 cm intermediate grids.	Exact counts and class totals; height statistics within explicit tolerance.
4. Bound working buffers	Reuse arrays, avoid unnecessary copies, release old scans and use a small queue.	Do not silently drop input or hide latency in a growing backlog.
5. Fix negative-obstacle work	Replace broad nested empty-cell scans with sensor-ray and visibility-based candidates.	A missing return is not proof of a pothole. Validate candidates.
6. Separate ground from obstacles	Estimate supported ground height, then obstacle height, slope and roughness.	Unknown ground stays unknown; sidewalk does not automatically mean drivable.
7. Resolve boundaries and labels	Use explicit cell ownership near ring boundaries; distinguish potentially moving objects from measured motion.	No overlapping ownership or misleading movement claims.
8. Add selective refinement later	Keep supported hazards detailed while simple terrain stays coarse, under a fixed cell budget.	Refinement adds cells. Verify net savings, misses and false alarms.

Do not optimize only for recall

If one noisy point marks a large square as an obstacle, recall can rise while the map becomes less usable. Measure **missed hazards and false alarms**, as well as overlap accuracy and ground-height error. Compare against a 2D baseline using the same semantic predictions; the existing height-threshold baseline is not equivalent.

A useful distinction

Smaller saved cells mainly reduce map memory. Direct construction and fewer copies target peak memory and processing time. A faster neural network mainly affects inference. Each change needs its own before/after measurement.

FPS: what it really means

One frame here means one LiDAR scan. Processing FPS is the number of completed scan-to-map results produced per second.

Time for one complete scan	Sequential capacity	Meaning
200 milliseconds	5 FPS	Too slow to consume every scan at a 10 Hz input rate.
100 milliseconds	10 FPS	Matches a 10 Hz input on average, with no spare time.
50 milliseconds	20 FPS	Can process recorded data faster than capture speed, with headroom.

Simple formula: capacity is approximately 1,000 / milliseconds per scan for sequential processing. With overlapping stages, measure completed scans / elapsed time directly; the reciprocal of one stage or median latency is not full-pipeline FPS.

Three different speeds must stay separate

Inference speed: how quickly the neural network predicts labels.
Pipeline speed and latency: reading/projecting a scan, predicting labels, building the map, computing required analytics, and delivering a fresh result.
Display refresh: how often the viewer redraws. Redrawing one old map 60 times does not mean 60 new maps per second.

KITTI was captured at **10 scans per second**, roughly one every 100 ms. A natural first target is sustained 10 Hz playback with no growing queue and low result age. A target such as p95 latency below 100 ms is an engineering objective to verify, not a guarantee of vehicle safety. [1]

Example: where speed comes from

Illustrative only: reading/projection 10 ms + inference 30 ms + grid 40 ms + analytics/display handoff 20 ms = 100 ms, or about 10 FPS sequential capacity. Reducing grid time to 10 ms gives 70 ms, or about 14.3 FPS. On 10 Hz playback you would still receive only 10 new scans per second, but have more headroom and fresher results.

What the project has already reported

Project documents report T4 model inference falling from **50.6 ms median at FP32 to 26.2 ms at FP16**, about 1.93 times faster for that stage. This is a historical documented measurement, not a new local result. The 26.2 ms figure does not establish 38 FPS for the complete system. [L3]

How to improve FPS using this code

Profile first. Then test direct cell construction, fewer array copies, bounded overlap of reading and processing, and efficient hazard analysis. The model already supports FP16; verify it on the intended GPU. Consider inference-mode execution or a compiled inference path only if profiling justifies it. Reduce model input resolution or use quantization later, with accuracy checks: these can erase small-object evidence. [3]

More sequences and a convincing demo

Use the larger dataset to demonstrate consistency. Use a short live demonstration to explain the benefit clearly.

There are 11 publicly labelled sequences, not 10

SemanticKITTI provides labels for **00-10 inclusive**. Sequences 11-21 form the test set and their labels are withheld. We currently have 09 and 10. We can add the remaining labelled sequences later, after checking storage. More saved data does not automatically improve the model or speed. [2]

Example: a library may hold 1,000 books, but you only need one open on your desk. Likewise, keep the dataset on disk and load the current scan plus a small bounded buffer. Running more sequences increases total evaluation time; it need not increase runtime memory with dataset length. Scene complexity still changes the working footprint.

For a per-scan or rolling local map, replace old results or evict cells outside the active area. If a persistent global map is added later, it can grow with travel and needs its own storage policy. Do not combine independent sequences into one global scene without registration.

Keep evaluation separate from demonstration

Use a held-out validation split and record which sequences the pretrained model saw during training. A replay of training data is not an unseen-data accuracy test. Freeze thresholds before final validation. Include representative hazards and failure cases in the demo; retain a per-sequence results table for broader evidence.

Live panel item	What the audience should see
Mode and input	Predicted labels or ground-truth replay; sequence, frame and sensor timestamp.
Memory	Current map MB, peak process RAM and peak GPU memory, clearly separated.
Speed	New completed maps/second, input rate, and p50/p95 result delay.
Reliability	Dropped scans, failed scans, queue depth and age of the newest result.
Quality	Measured hazard misses/false alarms and accuracy by distance, with evaluation provenance.

p95 example: p95 = 80 ms means 95% of measured results arrived within 80 ms under the stated timing definition. It is more informative than a single best frame. Measure capture/replay-release to map-ready age; label display/network delay separately when using VNC.

Two demonstration modes

Real-time replay: release scans at their recorded timestamps and show whether the system keeps up. **Unthrottled benchmark:** process the same stored scans as fast as possible to measure capacity. Run variants separately on the same hardware so comparison workloads do not distort the live result.

Novelty and the implementation plan

The project can become stronger without claiming that established mapping ideas are new.

Existing idea	Relevant earlier work	Implication
Near-fine / far-coarse maps	Montemerlo & Thrun, 2004. [4]	Distance-based resolution alone is established.
Semantic elevation mapping	SEL Map, 2022. [5]	Height plus semantic layers alone is established.
Multiresolution elevation	Schoppmann et al., 2021. [6]	Multiple elevation resolutions are established.
Semantic/geometric refinement	MAP-ADAPT, ECCV 2024. [7]	Refining important or complex regions is not automatically new.
Segmentation + adaptive elevation	RAEM, 2026; publisher abstract reviewed. [8]	A close overall direction already exists.

SalsaNext, sparse mapping structures, statistics-merging methods and negative-obstacle detection also have prior work. Your contribution today is the particular implementation and integration, including deterministic aggregation and preservation of class counts. We have **not established research novelty**; this search does not prove an identical full implementation exists. [9-12]

A candidate contribution to investigate

Can we preserve navigation-critical hazard evidence under a fixed memory budget? Define what must survive a cell merge, then design and evaluate that rule. For example, a supported curb or depression near the planned route could prevent merging while a flat patch is compressed. Extra grid cells cannot recover a hazard that the sensor never observed.

Build in this order

Step 1 - Establish the baseline: reproducible map memory, peak RAM, stage timings and accuracy on 09/10.
Step 2 - Compact the existing grid: smaller records, controlled numerical error and unchanged class counts.
Step 3 - Remove waste: direct construction and bounded buffers; measure full-pipeline speed.
Step 4 - Improve meaning: ground/obstacle separation, unknown space and validated hazard candidates.
Step 5 - Demonstrate and expand: connect live predictions, add counters, then evaluate across additional sequences.
Step 6 - Investigate the differentiator: compare a budgeted hazard-preserving rule with distance-only and semantic/geometric alternatives.

What would make the case convincing?

The same useful map information, lower measured memory, sustained processing on stated hardware, and fewer missed hazards without excessive false alarms. These are outcomes to demonstrate. We should not promise 10x memory reduction, a particular FPS increase, or guaranteed safety before testing.

Evidence, definitions and references

Use this page to distinguish what was measured, what was recorded earlier, and what is still proposed.

Measured in our earlier local check: the 10-frame memory sample (5.03 / 3.86 / 1.31 MB).
Documented historical results: full-sequence benchmark and T4 inference timings; not reproduced locally for this report.
Proposed: 56-byte cell layout, direct construction, speed improvements, budgeted refinement and a larger evaluation.
Not yet proved: full-pipeline demo FPS, peak-memory savings after optimization, dependable pothole detection, or research novelty.

Local evidence and reproducibility

L1 - Exploratory sample: sequence 09 frames 000000, 000397, 000795, 001193, 001590; sequence 10 frames 000000, 000300, 000600, 000900, 001200. Ground-truth labels; all points with planar radius 2.5-100 m; full derived statistics. Uniform comparison: one 5 cm band over the same range. Mean occupied cells: 45,995.2 foveated and 59,854.5 uniform. MB means 1,000,000 bytes. Sample outputs were recorded in this conversation; a persistent benchmark artifact remains to be added.

L2 - Implementation: grid/foveated_grid.py, analytics/cell_stats.py, analytics/negative_obs.py, offline/baselines.py and benchmark/harness.py under src; configs/rings.yaml; scripts/viz_dual_fovea.py. See docs/stages/stage-7-benchmark-pareto.md for the historical benchmark.

L3 - Timing: docs/stages/stage-5-gpu-latency.md reports the T4 FP32/FP16 results. The current local viewer uses provided labels, and model weights were not present during this review. File paths above are relative to the project repository.

Primary sources - clickable references

1. KITTI sensor setup: 10 scans per second
2. SemanticKITTI: labelled sequences, formats and dataset split
3. PyTorch performance tuning guide
4. Montemerlo & Thrun: Multi-Resolution Pyramid (2004)
5. SEL Map: semantic terrain and elevation mapping (2022)
6. Multi-Resolution Elevation Mapping and Safe Landing (2021)
7. MAP-ADAPT: adaptive semantic 3D maps (ECCV 2024)
8. Resolution-adaptive elevation mapping / RAEM (2026)
9. SalsaNext: LiDAR semantic segmentation (2020)
10. OctoMap: sparse multiresolution occupancy mapping
11. Chan et al.: algorithms for sample variance (1983)
12. Larson & Trivedi: off-road negative obstacles (2011)

Plain-language glossary

Segmentation: assigning a class to each point. **Variance:** how much heights differ within a cell; it is not automatically ground roughness. **Recall:** proportion of reference hazards found. **Precision:** proportion of reported hazards that are correct. **Latency:** delay for one result. **Throughput/FPS:** completed results per second.